

To-Do List App: Svelte Frontend Architecture & Code Structure

Overview

This document provides a detailed description of the architecture, main components, features, state management, and dependencies of the Svelte-based To-Do List web application found in the `todo_app_frontend` container. This application is designed with modern web standards, supporting user authentication, CRUD operations for tasks, filtering and sorting, and a responsive, minimalistic UI.

1. Architecture

1.1. Application Structure

The application is implemented as a SvelteKit project, using modular Svelte components for UI and TypeScript for type safety. The client communicates with a backend API (assumed to be a RESTful API) over HTTP, primarily for user authentication and CRUD operations on tasks.

The main directory layout:

```
todo_app_frontend/
├── src/
│   ├── routes/
│   │   ├── +page.svelte           // Main application page
│   │   ├── +layout.svelte        // Application shell/layout
│   │   ├── AuthPanel.svelte      // User authentication controls
│   │   ├── Counter.svelte        // Demo component (optional utility)
│   │   ├── Header.svelte         // Top navigation and theme toggle
│   │   ├── Sidebar.svelte        // Sidebar for filtering/sorting tasks
│   │   ├── TaskForm.svelte       // Add/edit task form
│   │   ├── TaskList.svelte       // Lists tasks. supports edit/delete co
│   │   └── about/                // Static about page
│   └── stores/                   // Svelte stores for app state
├── app.css                      // Global stylesheet
├── app.d.ts                     // Type definitions
├── app.html                     // Main HTML template
├── static/                      // Static assets (e.g., favicon)
├── svelte.config.js             // SvelteKit config
├── vite.config.ts               // Vite (dev & build tool) config
└── package.json                 // Dependency and script config
```

1.2. Application Flow & Data Lifecycle

- **Startup:** Loads UI layout, checks for persisted user session in `localStorage`, and fetches current user's tasks if authenticated.
 - **Authentication:** All user authentication is handled through `AuthPanel.svelte` using API calls to `/api/auth/login` and `/api/auth/signup`. Auth state is stored in a Svelte store (`stores/auth.ts`).
 - **CRUD Operations:** Task-related operations are performed via HTTP requests to `/api/tasks` endpoints, with authorization headers from logged-in users.
 - **State Management:** Uses Svelte writable stores for authentication, task list, task filters/sorting, and currently editing task.
 - **UI Reactivity:** UI is composed of reactive Svelte components, each responsible for a logical slice of the UI and subscribing to relevant stores.
 - **Filter & Sort:** Filtering (active/completed/all) and sorting (by due date, created date, alphabetical) are managed via store-driven state.
-

2. Main Features

- **User Authentication:** Sign up, sign in, and log out using email and password. Session is persisted in `localStorage` and restored on reload.
 - **Task Creation:** Add new tasks with optional due date.
 - **Edit Tasks:** Edit tasks inline via a dedicated form.
 - **Delete Tasks:** Confirm and remove tasks.
 - **Mark Complete:** Toggle task completion status.
 - **Task Filters:** Filter by all/active/completed.
 - **Task Sorting:** Sort by due date (asc/desc), creation date (asc/desc), or alphabetically (A-Z, Z-A).
 - **Responsive UI:** Modern, minimal design adapts for mobile/desktop.
 - **Theme Support:** Light/Dark theme toggle with persistence (saved in `localStorage`).
 - **Persistent State:** User session and task list persist between reloads when authenticated.
-

3. Svelte Component Structure

Below is a description of the main Svelte components and their roles:

- **+layout.svelte**
App shell; includes header, main content, and footer. Loads the common stylesheet. Wraps routed components.

- **+page.svelte**
Main to-do interface. Lays out:
 - Header (title, AuthPanel)
 - Sidebar (filters/sorting)
 - Task section (TaskForm, TaskList, or login prompt)
 - Footer
- **AuthPanel.svelte**
Login/signup flow and displays current user. Operates via Svelte store.
 - Shows login or signup form if logged out
 - Shows user details and logout button if logged in
- **Sidebar.svelte**
Let users filter and sort tasks.
 - Buttons for All/Active/Completed
 - Dropdown for sort order
- **TaskForm.svelte**
Form for adding a new task or editing an existing one.
 - Binds to the `taskEdit` store to toggle between add/edit mode
 - Inputs for description and due date
- **TaskList.svelte**
Displays the current (filtered and sorted) list of tasks.
 - Each task: checkbox for complete, edit & delete actions
 - Reacts to changes in tasks, filter, or sort
- **Header.svelte**
Navigation bar with theme toggle button. Handles switching and persisting theme preference.
- **about/+page.svelte**
Static about page.
- **Store files**
(`stores/auth.ts` , `stores/tasklist.ts` , `stores/taskfilters.ts`)
Centralized state for user, task list, filters, and sorting.
 - `auth.ts` : user state, login/signup/logout handlers
 - `taskfilters.ts` : filter and sort settings
 - `tasklist.ts` : CRUD for tasks, manages main task list and edit mode

4. State Management

Svelte writable stores are used for all shared state.

4.1. Stores Used

- `user : {id, email} | null` - Current authenticated user or null
Defined in: `stores/auth.ts`
- `tasks : Task[]` - List of tasks for current user
Defined in: `stores/tasklist.ts`
- `taskEdit : Task | null` - Task under edit, if any
Defined in: `stores/tasklist.ts`
- `filter : 'all' | 'active' | 'completed'` - Current filter
Defined in: `stores/taskfilters.ts`
- `sortBy : string` - Current sort order
Defined in: `stores/taskfilters.ts`

When user or filter/sort changes, relevant stores auto-update and trigger fetching/filtering/sorting logic and corresponding UI updates.

5. Key Dependencies

All dependencies are specified in `package.json` :

- **Framework & UI:**
 - `svelte` & `@sveltejs/kit` : Core framework and app runtime
 - `@sveltejs/adapter-auto` : Platform adapter
 - **Tooling & Build:**
 - `vite`, `typescript`, `eslint`, `eslint-plugin-svelte`
 - `@testing-library/svelte`, `vitest`, `jsdom` : Testing utilities
 - **Stylina:**
 - `@fontsource/fira-mono` : Font for code/monospace
 - **Others:**
 - `globals`, `@eslint/js`, `@eslint/compat` : Config/test utilities
-

6. Code Structure

- **Global Styles** in `src/app.css` (uses CSS variables, supports themes)
 - **Responsive Layout:** All main UI defined by Svelte components, responsive breakpoints set in styles.
 - **TypeScript:** Used across logic and Svelte components for safety.
 - **API Integration:** `fetch` is used for all network calls; endpoints are relative (proxied by Vite config to a backend server).
 - **LocalStorage:** Utilized for user session and theme persistence.
-

7. Application Structure Diagram

```
flowchart TD
    A(Main App [`${+layout.svelte`])
    A --> B(Main Page [`${+page.svelte`])
    B --> C[Header]
    B --> D[Sidebar]
    B --> E[AuthPanel]
    B --> F[TaskForm]
    B --> G[TaskList]
    D <-- H[taskfilters store]
    F <-- I[tasklist store]
    G <-- I[tasklist store]
    E <-- J[auth store]
    A --> K(Footer)
    B --> L(About Page [`${about/+page.svelte`])
```

Legend:

- Rectangles: UI Components
 - Parallelograms: Svelte stores
-

8. HTTP API and Backend Interface

- All authenticated network requests use `/api/tasks` and `/api/auth/*` endpoints.
 - Auth state is passed in Authorization headers.
 - Backend is proxied locally to `http://localhost:8000` for `/api/` routes via Vite config.
-

9. Theming and Styling

- **Colors:**
 - Primary: `#2d8cff`
 - Secondary: `#e0e7ef`
 - Accent: `#ffbe2d`
 - Both Light and Dark themes are supported and toggleable.
 - CSS custom properties used extensively for theme management.
-

10. Responsive Design

- Designed to be usable both on desktop and mobile.
 - Media queries adjust layouts for screens `<900px` and `<600px`, stacking sidebar and task form vertically, resizing UI components as appropriate.
-

11. Extensibility

- Additional routes, pages, or features (like reminders/notifications, task categories, or sharing) can be added as new Svelte components or stores.
 - Authentication logic can be swapped out or extended to support OAuth or other providers.
 - State management can be further modularized for larger projects if necessary.
-

12. Project Scripts

Available npm scripts in package.json : - `npm run dev` — Start dev server - `npm run build` — Build for production - `npm run preview` — Preview built app - `npm run lint` — Lint code - `npm run test:unit` — Run unit tests - `npm run test` — Run all tests

13. Useful References

- [SvelteKit Documentation](#)
 - [Svelte Official Site](#)
 - [Vite Documentation](#)
-

This documentation is intended to provide a comprehensive technical understanding to new maintainers and contributors to the To-Do List Svelte frontend.